

Confronto tra Tecnologie Big Data per l'Analisi di Dataset di Grandi Dimensioni

Francesco Pittacolo

<https://github.com/francesco-pittacolo/big-data-flight-analysis>

1 Introduzione

Il presente progetto ha l'obiettivo di confrontare diverse tecnologie per l'analisi di Big Data, applicate a un dataset reale di grandi dimensioni. In particolare, l'attenzione è rivolta alla progettazione delle elaborazioni, alla preparazione e alla pulizia dei dati, all'implementazione delle analisi e alla valutazione delle prestazioni in termini di efficienza e scalabilità.

Il caso di studio considerato è il *Flight Delay Dataset 2024*, disponibile sulla piattaforma Kaggle, che contiene oltre 7 milioni di record e circa 35 attributi relativi a voli commerciali negli Stati Uniti. Il dataset include informazioni su compagnie aeree, aeroporti di partenza e arrivo, ritardi, cancellazioni e altri indicatori operativi.

Le analisi sono state implementate utilizzando Apache Hive, Spark Core e Spark SQL, al fine di confrontarne espressività e prestazioni.

Un ulteriore obiettivo del progetto è lo studio delle prestazioni delle soluzioni realizzate in differenti contesti esecutivi. Per questo motivo, tutte le sperimentazioni sono state condotte sia in ambiente locale sia in ambiente cluster, così da valutare l'impatto della distribuzione del carico computazionale sui tempi di esecuzione e sulla scalabilità delle soluzioni.

Infine, sono state eseguite analisi su dataset di dimensioni crescenti, ottenuti a partire da quello originale, con l'obiettivo di studiare il comportamento delle diverse tecnologie al variare del volume dei dati.

2 Preparazione dei dati

Prima di procedere con le analisi, il dataset è stato sottoposto a una fase di preparazione finalizzata a garantirne la consistenza e a renderlo idoneo alle elaborazioni successive. Le operazioni sono state implementate mediante PySpark, sfruttando le funzionalità di Apache Spark per la trasformazione e la gestione dei dati.

2.1 Caricamento del dataset

Il dataset originale in formato CSV è stato caricato tramite Spark DataFrame API, utilizzando l'inferenza automatica dello schema. Il caricamento è stato parametrizzato per supportare due modalità di esecuzione:

- **Modalità locale:** il dataset viene letto dal filesystem locale;
- **Modalità cluster:** il dataset viene letto da uno storage distribuito basato su Amazon S3.

2.2 Verifica della consistenza dei dati

Prima della definizione della pipeline di preprocessing è stata effettuata una fase preliminare di validazione del dataset, finalizzata all'individuazione di possibili inconsistenze nei principali attributi temporali. L'analisi ha evidenziato la presenza di 128 record su oltre 6.9 milioni di osservazioni (meno dello 0.002%) con possibili anomalie, individuate confrontando i valori temporali memorizzati con quelli ricostruibili a partire dagli attributi correlati. Data la loro incidenza estremamente ridotta rispetto alla dimensione complessiva del dataset, tali anomalie possono avere un impatto trascurabile sulle statistiche aggregate calcolate nelle analisi successive. Per questo motivo si è deciso di non applicare ulteriori operazioni di filtraggio, al fine di preservare la rappresentatività del dataset in uno scenario realistico.

È stata poi verificata la coerenza dell'attributo relativo al mese del volo, controllando che tutti i valori presenti rispettassero il dominio atteso (1–12). La verifica non ha evidenziato alcun record con valori fuori dall'intervallo previsto, confermando la correttezza del campo rispetto al formato atteso.

L'analisi esplorativa ha inoltre evidenziato la presenza di diverse tipologie di volo, tra cui voli cancellati e voli dirottati. In particolare, i voli cancellati (`cancelled = 1`) rappresentano circa l'1.36% del totale delle osservazioni, mentre i voli dirottati (`diverted = 1`) circa lo 0.25%.

Nei voli cancellati è stato osservato che una piccola percentuale di record (circa il 3%) presenta valori non nulli nel campo relativo al ritardo in partenza. Poiché tali osservazioni non rappresentano voli effettivamente completati, questi valori non sono stati considerati affidabili ai fini dell'analisi dei ritardi e sono stati quindi uniformati a `NULL` durante la fase di preprocessing, garantendo maggiore coerenza semantica del dataset.

Per quanto riguarda i voli dirottati, tutti i record presentano valori validi per il ritardo in partenza, mentre il ritardo in arrivo risulta sistematicamente non valorizzato, in quanto il volo non raggiunge la destinazione originaria. Poiché tali voli risultano comunque effettivamente operati, i valori di ritardo in partenza sono stati mantenuti validi e i record sono stati conservati senza ulteriori modifiche.

2.3 Analisi dei valori mancanti

Dopo la verifica della consistenza dei dati, è stata condotta un'analisi della presenza di valori nulli nelle colonne del dataset, al fine di individuare eventuali problemi di completezza dei dati e valutare le strategie di gestione più appropriate.

L'analisi ha evidenziato che alcune colonne presentano valori mancanti concentrati in particolari tipologie di volo. In particolare, l'attributo `cancellation_code` presenta circa il 98.6% di valori nulli, comportamento atteso in quanto il campo viene valorizzato esclusivamente per i voli cancellati.

Analogamente, alcune metriche operative quali `arr_delay`, `dep_delay`, `air_time`, `wheels_off` e `wheels_on` presentano valori mancanti compresi tra l'1.3% e l'1.6% delle osservazioni. Tali valori risultano coerenti con la presenza di voli cancellati, per i quali le informazioni relative all'effettiva operatività del volo non sono disponibili.

Le colonne utilizzate come chiavi di aggregazione nelle analisi successive, come `op_unique_carrier`, `origin` e `dest`, non presentano valori nulli, garantendo la possibilità di effettuare correttamente le operazioni di grouping e aggregazione.

Sulla base di queste osservazioni, non è stata applicata una rimozione indiscriminata di righe con valori mancanti, ma sono state effettuate esclusivamente operazioni di gestione mirate in base al significato semantico degli attributi.

2.4 Pulizia e normalizzazione dei dati

La fase di preprocessing è stata eseguita tramite Spark con l'obiettivo di uniformare la struttura del dataset e renderlo adatto alle successive elaborazioni distribuite, preservando il più possibile le informazioni originali.

Le principali operazioni applicate sono state:

- normalizzazione degli attributi categoriali tramite conversione in maiuscolo e rimozione degli spazi iniziali e finali, al fine di garantire uniformità nei valori utilizzati durante le operazioni di raggruppamento;
- conversione esplicita dei principali attributi numerici nei rispettivi tipi di dato, in particolare mese, variabili di cancellazione e dirottamento e ritardi di partenza e arrivo, evitando possibili inconsistenze durante le successive aggregazioni;
- gestione dei voli cancellati: per tali record i valori relativi ai ritardi di partenza e arrivo sono stati impostati a `NULL`, in quanto il volo non è stato completato e tali informazioni non rappresentano misure effettive;
- gestione dei voli dirottati: per questi record il ritardo in partenza è stato conservato, mentre il ritardo in arrivo è stato impostato a `NULL`, poiché la destinazione originaria non è stata raggiunta.

2.5 Gestione della scalabilità del dataset

Per valutare la scalabilità delle tecnologie utilizzate, è stata generata una serie di dataset di dimensione crescente a partire dal dataset originale.

In particolare, oltre al dataset di partenza (processato), sono state adottate due ulteriori strategie:

- **campionamento casuale** per ottenere dataset ridotti (25%, 50%, 75%);
- **replica controllata** del dataset per ottenere versioni di dimensioni maggiori (200%, 400%).

Il campionamento è stato eseguito tramite la funzione `sample` di Apache Spark, mentre la generazione dei dataset di dimensione maggiore è stata realizzata tramite replica controllata del dataset originale mediante operazioni di unione.

Quest'ultima operazione non introduce nuove informazioni semantiche, ma consente esclusivamente di simulare scenari con quantità di dati crescenti, permettendo di valutare il comportamento delle tecnologie al variare del carico elaborativo.

2.6 Formato di output

I dataset processati sono stati salvati in formato Parquet, al fine di migliorare le prestazioni delle successive elaborazioni. Il formato colonnare consente infatti una migliore compressione e un accesso più efficiente alle colonne rilevanti per le analisi.

Il percorso di salvataggio è stato configurato in base alla modalità di esecuzione: nelle esecuzioni locali i dataset sono stati salvati su HDFS, mentre nelle esecuzioni in ambiente cluster sono stati memorizzati su Amazon S3.

2.7 Implementazione delle analisi

A partire dal dataset preprocessato e dalle relative versioni scalate, sono state realizzate le analisi richieste dalla traccia del progetto, con l'obiettivo di valutare il comportamento delle principali entità operative del sistema di trasporto aereo.

Le diverse implementazioni sono state sviluppate utilizzando Spark Core, Spark SQL e Hive, al fine di confrontarne espressività, semplicità implementativa ed efficienza in scenari con volumi di dati crescenti. I risultati di ciascuna analisi sono stati salvati su HDFS in locale e su S3 in ambiente distribuito.

3 Analisi dei dati

3.1 Statistiche delle compagnie aeree

La prima analisi ha l'obiettivo di caratterizzare il comportamento operativo delle compagnie aeree presenti nel dataset, in termini di rotte servite, ritardi e tasso di cancellazione.

3.1.1 Preparazione dei dati

Per l'esecuzione del job è stato utilizzato il seguente sottoinsieme di attributi:

- identificativo della compagnia aerea (`op_unique_carrier`);
- aeroporti di origine e destinazione (`origin`, `dest`);
- ritardo in arrivo (`arr_delay`);
- indicatore di cancellazione (`cancelled`);
- mese del volo (`month`).

3.1.2 Descrizione dell'analisi

L'obiettivo dell'analisi è caratterizzare l'operatività delle compagnie aeree sulle diverse tratte servite, fornendo informazioni sia sulla puntualità dei voli sia sulla continuità del servizio.

A tal fine, per ogni coppia composta da compagnia aerea e tratta (`origin-dest`), sono state calcolate le seguenti metriche:

- numero totale di voli operati;
- ritardo minimo, massimo e medio in arrivo;
- tasso di cancellazione dei voli;
- insieme dei mesi nei quali la tratta risulta operativa.

Le statistiche ottenute consentono di confrontare le prestazioni delle compagnie sulle diverse rotte, evidenziando sia il comportamento medio dei ritardi sia eventuali criticità legate alla regolarità del servizio.

L'output generato consiste in una tabella aggregata in cui ogni riga rappresenta una specifica combinazione compagnia-tratta e contiene le metriche precedentemente descritte.

Il risultato è rappresentato dalla seguente struttura dati:

```
root
|-- op_unique_carrier: string
|-- route: string
|-- num_flights: long
|-- min_arr_delay: double
|-- max_arr_delay: double
|-- avg_arr_delay: double
|-- cancel_rate: double
|-- months: array
|   |-- element: integer
```

3.1.3 Implementazione con Spark Core

L'implementazione tramite Spark Core prevede inizialmente la lettura del dataset in formato Parquet e la conversione dei dati in un RDD, sul quale vengono applicate le trasformazioni necessarie per costruire le statistiche richieste.

Ogni record viene trasformato tramite una funzione di mapping in una coppia chiave-valore, dove la chiave è composta dall'identificativo della compagnia aerea `op_unique_carrier` e dalla tratta, ottenuta concatenando gli aeroporti di origine e destinazione (`origin-dest`).

Per ciascuna chiave viene mantenuto un accumulatore contenente le informazioni necessarie al calcolo delle metriche finali:

- numero totale di voli;
- minimo e massimo ritardo in arrivo osservato;
- somma dei ritardi e numero di valori validi per il calcolo della media;
- numero di voli cancellati;
- insieme dei mesi in cui la tratta risulta operativa.

L'aggregazione è stata realizzata tramite l'operazione `aggregateByKey`, che permette di combinare i dati in due fasi. Nella prima fase, attraverso la funzione di aggregazione locale, ogni record aggiorna l'accumulatore relativo alla propria chiave all'interno della partizione in cui viene elaborato. Successivamente, dopo la fase di shuffle, gli accumulatori parziali prodotti dalle diverse partizioni vengono combinati tramite una funzione di merge, ottenendo così un unico accumulatore per ciascuna combinazione compagnia-tratta.

Nella fase finale viene applicata una funzione di finalizzazione che calcola le metriche derivate, tra cui il ritardo medio in arrivo e il tasso di cancellazione, e converte l'insieme dei mesi raccolti nella lista finale dei mesi di operatività.

Il risultato viene quindi convertito in un DataFrame Spark, ordinato per identificativo della compagnia aerea e tratta, e infine salvato in formato Parquet per le successive fasi di analisi.

3.1.4 Implementazione con Spark SQL

L'implementazione tramite Spark SQL è stata realizzata utilizzando le API DataFrame di Spark. Il dataset viene inizialmente letto in formato Parquet e rappresentato come DataFrame, permettendo di esprimere le operazioni di trasformazione e aggregazione in modo dichiarativo.

Come prima operazione viene costruita una nuova colonna **route**, ottenuta concatenando gli aeroporti di origine e destinazione (**origin-dest**). Successivamente, il DataFrame viene raggruppato per compagnia aerea e tratta (**op_unique_carrier, route**), generando una riga aggregata per ciascuna combinazione.

Per ogni gruppo vengono calcolate le metriche richieste:

- numero totale di voli operati;
- ritardo minimo, massimo e medio in arrivo;
- tasso di cancellazione dei voli;
- insieme ordinato dei mesi nei quali la tratta risulta operativa.

Le metriche vengono calcolate attraverso le funzioni di aggregazione messe a disposizione da Spark, utilizzando operazioni come **count**, **min**, **max**, **avg**, **sum** e **collect_set**. Queste permettono di ottenere rispettivamente il numero di voli, i valori minimo, massimo e medio del ritardo in arrivo, il tasso di cancellazione e l'insieme dei mesi in cui la tratta risulta operativa.

Infine, il risultato viene ordinato per identificativo della compagnia aerea e tratta e salvato in formato Parquet.

3.1.5 Implementazione con Hive

L'implementazione in Hive segue un approccio dichiarativo basato su SQL, in cui le operazioni di trasformazione e aggregazione vengono espresse tramite query eseguite dal motore Hive.

In una prima fase viene definita una tabella esterna di input che punta ai dati preprocessati salvati in formato Parquet. Tale tabella consente di rendere disponibili le informazioni all'interno dell'ambiente Hive mantenendo invariato il formato originale dei dati.

Successivamente viene creata la tabella finale contenente le statistiche aggregate a livello di compagnia aerea e tratta. La tratta viene ottenuta concatenando gli aeroporti di origine e destinazione (**origin-dest**), mentre il raggruppamento viene effettuato sugli attributi **op_unique_carrier**, **origin** e **dest**.

Per ciascuna combinazione compagnia-tratta vengono calcolate le metriche richieste:

- numero totale di voli operati tramite **COUNT**;
- ritardo minimo e massimo in arrivo tramite **MIN** e **MAX**;
- ritardo medio in arrivo tramite **AVG**;
- tasso di cancellazione tramite il rapporto tra il numero di voli cancellati e il numero totale di voli;
- insieme ordinato dei mesi di operatività della tratta tramite **COLLECT_SET** e **SORT_ARRAY**.

L'aggregazione viene quindi eseguita direttamente tramite la clausola **GROUP BY**, producendo una tabella finale in formato Parquet con una riga per ogni coppia compagnia-tratta. Il risultato viene inoltre ordinato per identificativo della compagnia aerea e tratta tramite la clausola **ORDER BY**.

Infine, la tabella esterna utilizzata per accedere ai dati di input viene eliminata al termine dell'elaborazione.

3.1.6 Risultato del job

Di seguito nella Tabella 1 si riportano le prime 10 righe del risultato prodotto dal job. Poiché le tre implementazioni (Spark Core, Spark SQL e Hive) generano lo stesso schema e la stessa struttura logica dell'output, viene mostrato un unico esempio rappresentativo.

Carrier	Route	Flights	Min Arr	Max Arr	Avg Arr	Cancel Rate	Months
9E	ABE-ATL	1017	-47.0	575.0	0.67	0.0138	[1,2,3,4,5,6,7,8,9,10,11,12]
9E	ABY-ATL	38	-32.0	321.0	4.37	0.0	[1,2,3,11,12]
9E	AEX-ATL	878	-34.0	979.0	9.61	0.0103	[1,2,3,4,5,6,7,8,9,10,11,12]
9E	AGS-ATL	1580	-36.0	1091.0	6.89	0.0209	[1,2,3,4,5,6,7,8,9,10,11,12]
9E	AGS-AUS	2	12.0	92.0	52.0	0.0	[4]
9E	AGS-DCA	2	21.0	146.0	83.5	0.0	[4]
9E	AGS-DTW	6	-15.0	9.0	-3.17	0.0	[4]
9E	AGS-JFK	2	-3.0	44.0	20.5	0.0	[4]
9E	AGS-LGA	11	-17.0	43.0	-1.0	0.0	[4]
9E	ALB-DTW	435	-51.0	394.0	-4.31	0.0046	[1,2,3,4,5,6,7,8,9,10,11,12]

Tabella 1: Esempio di statistiche per compagnia aerea e tratta

3.2 Report dei ritardi per aeroporto e periodo temporale

La seconda analisi ha l'obiettivo di costruire un report dettagliato dei ritardi in partenza per ciascun aeroporto e mese, introducendo una suddivisione per fasce di ritardo e un'analisi delle principali cause associate ai ritardi e alle cancellazioni.

3.2.1 Preparazione dei dati

Per l'esecuzione del job sono stati utilizzati i seguenti attributi del dataset preprocessato:

- aeroporto di origine (**origin**);
- mese del volo (**month**);
- ritardo in partenza (**dep_delay**);
- ritardo in arrivo (**arr_delay**);
- indicatore di cancellazione (**cancelled**);
- codice di cancellazione (**cancellation_code**);
- cause di ritardo (**carrier_delay**, **weather_delay**, **nas_delay**, **security_delay**, **late_aircraft_delay**).

Tali attributi sono stati considerati sufficienti per la costruzione delle metriche richieste.

3.2.2 Descrizione dell'analisi

L'analisi considera i voli con ritardo in partenza positivo per la costruzione delle fasce di ritardo. I voli puntuali o partiti in anticipo non vengono quindi inclusi nelle categorie di ritardo, mentre i voli cancellati vengono rappresentati separatamente tramite una categoria dedicata, utilizzata per analizzare le relative cause di cancellazione.

Per ciascun aeroporto di partenza e per ciascun mese, i voli con ritardo sono stati suddivisi in tre fasce di ritardo in partenza:

- *low*: ritardo inferiore a 15 minuti;
- *medium*: ritardo compreso tra 15 e 60 minuti;
- *high*: ritardo superiore a 60 minuti.

Per ciascuna combinazione aeroporto-mese-fascia sono state calcolate le seguenti metriche:

- numero totale di voli;
- ritardo medio in partenza;
- ritardo medio in arrivo;
- le tre cause più frequenti associate a ritardi o cancellazioni.

Il risultato finale è organizzato in una struttura indicizzata per aeroporto, mese e fascia, in cui ciascun gruppo contiene le statistiche aggregate e le principali cause osservate.

Il risultato dell'analisi è rappresentato dalla seguente struttura dati:

```
root
|-- origin: string
|-- month: integer
|-- band: string
|-- num_flights: long
|-- avg_dep_delay: double
|-- avg_arr_delay: double
|-- top_causes: array
|   |-- element: struct
|   |   |-- cause: string
|   |   |-- cause_count: long
```

3.2.3 Implementazione con Spark Core

In questa implementazione la logica principale consiste nell'associare ogni volo a una chiave composta da (*origin*, *month*, *band*), dove la fascia viene determinata in base al valore di *dep_delay* per i voli con ritardo, oppure impostata a *cancelled* nel caso di voli cancellati.

I voli con ritardo vengono classificati nelle fasce *low*, *medium* e *high*, mentre per i voli cancellati la causa viene ricavata dal campo *cancellation_code* e convertita nella relativa categoria (*carrier*, *weather*, *nas*, *security*).

Durante la fase di mapping, ogni record viene trasformato in una coppia chiave-valore. La chiave identifica la combinazione di aeroporto, mese e fascia a cui appartiene il volo, mentre il valore contiene le informazioni necessarie al calcolo delle metriche finali:

- somma dei ritardi in partenza;
- numero di valori validi per il calcolo del ritardo medio in partenza;
- somma dei ritardi in arrivo;
- numero di valori validi per il calcolo del ritardo medio in arrivo;
- numero totale di voli appartenenti al gruppo;
- dizionario contenente le frequenze delle cause osservate.

Per i voli non cancellati, le cause vengono individuate analizzando i campi `carrier_delay`, `weather_delay`, `nas_delay`, `security_delay` e `late_aircraft_delay`. Una causa viene aggiunta al dizionario solo quando il valore associato è maggiore di zero. Questo permette di rappresentare correttamente i casi in cui un singolo volo presenta più cause contemporaneamente.

La fase di aggregazione viene eseguita tramite `reduceByKey()`, che combina tutti i valori associati alla stessa chiave. Durante questa fase vengono aggiornate le somme necessarie al calcolo delle medie, il numero totale di voli e le frequenze delle cause individuate nei voli appartenenti allo stesso gruppo.

Successivamente, nella fase di finalizzazione, i valori aggregati vengono convertiti nelle metriche richieste:

- numero totale di voli;
- ritardo medio in partenza;
- ritardo medio in arrivo;
- prime tre cause più frequenti.

Le cause principali vengono ottenute ordinando le cause presenti nel dizionario associato a ciascun gruppo in base alla frequenza osservata in ordine decrescente e selezionando le prime tre. In caso di frequenze uguali viene utilizzato un ordinamento alfabetico per rendere il risultato deterministico.

Infine, il risultato viene convertito in un `DataFrame`, ordinato secondo aeroporto, mese e fascia di ritardo, e salvato in formato Parquet.

3.2.4 Implementazione con Spark SQL

L'implementazione tramite Spark SQL riprende l'utilizzo del modello `DataFrame` introdotto nella Task 3.1, sfruttando le operazioni di trasformazione e aggregazione fornite da Spark per generare il report richiesto.

La pipeline viene suddivisa principalmente nella costruzione delle statistiche sui voli e nell'analisi delle cause di ritardo e cancellazione.

Nella prima fase, i voli non cancellati con ritardo positivo vengono filtrati e viene aggiunta la colonna `band` tramite una funzione condizionale `when`. In base al valore di `dep_delay`, ogni volo viene assegnato alla fascia *low*, *medium* o *high*. Su questo insieme vengono calcolate le statistiche aggregate per aeroporto, mese e fascia tramite funzioni come `groupBy`, `count` e `avg`.

Parallelamente, i voli cancellati vengono elaborati separatamente assegnando alla colonna **band** il valore *cancelled*. La causa della cancellazione viene ricavata dal campo **cancellation_code** attraverso trasformazioni condizionali.

Per individuare le principali cause di ritardo, le colonne relative alle diverse tipologie di ritardo vengono raccolte in un array temporaneo e successivamente trasformate tramite **explode**. Questa operazione permette di ottenere una rappresentazione a righe delle cause effettivamente presenti, consentendo di gestire correttamente anche i casi in cui uno stesso volo presenta più cause contemporaneamente e facilitandone il conteggio tramite operazioni di aggregazione.

Le cause dei voli in ritardo e dei voli cancellati vengono successivamente unite tramite **unionByName** e aggregate per aeroporto, mese, fascia e causa, ottenendo il numero di occorrenze per ciascuna causa. Per selezionare le tre cause più frequenti viene utilizzata una window function con **row_number()**, che permette di ordinare le cause all'interno di ciascuna combinazione (**origin**, **month**, **band**) in base alla frequenza osservata. In caso di parità viene utilizzato, come nell'implementazione con Spark Core, un ordinamento alfabetico per rendere il risultato deterministico.

Infine, le statistiche sui voli e le informazioni sulle cause vengono unite tramite una **join**. Il risultato finale viene ordinato per aeroporto, mese e fascia e salvato in formato Parquet.

3.2.5 Implementazione con Hive

Nella prima fase viene creata una tabella intermedia contenente le statistiche aggregate per aeroporto, mese e fascia. La fascia (*low*, *medium*, *high*, *cancelled*) viene determinata tramite una clausola **CASE**, considerando il valore di **dep_delay** per i voli con ritardo e il flag di cancellazione per i voli cancellati. Per ciascun gruppo vengono calcolati il numero totale di voli e i ritardi medi in partenza e in arrivo tramite le funzioni di aggregazione **COUNT** e **AVG**.

Successivamente viene costruita una seconda tabella contenente il conteggio delle cause associate ai ritardi e alle cancellazioni. Per i voli non cancellati con ritardo positivo, le diverse tipologie di causa vengono estratte separatamente tramite query dedicate e successivamente unite mediante **UNION ALL**. Questa operazione permette di trasformare i diversi attributi delle cause in una rappresentazione a righe, nella quale ogni occorrenza può essere conteggiata indipendentemente dalle altre.

Per i voli cancellati, invece, la causa viene ricavata direttamente dal campo **cancellation_code** tramite una clausola **CASE**, associando ciascun codice alla relativa categoria.

Le occorrenze delle cause vengono quindi aggregate tramite **GROUP BY** per ciascuna combinazione aeroporto-mese-fascia-causa, ottenendo la frequenza associata a ciascuna causa di ritardo o cancellazione. La selezione delle tre cause più frequenti viene effettuata tramite **ROW_NUMBER() OVER**, che assegna un indice alle cause in base al numero di occorrenze all'interno di ciascun gruppo. In caso di parità viene utilizzato l'ordinamento alfabetico per rendere il risultato deterministico. Le cause selezionate vengono infine raccolte nella struttura finale tramite **COLLECT_LIST** e **NAMED_STRUCT**.

Infine, la tabella contenente le statistiche sui voli viene unita con quella contenente le principali cause tramite un'operazione di **JOIN**. Il risultato finale viene ordinato per aeroporto, mese e fascia e salvato in formato Parquet.

3.2.6 Risultato del job

Di seguito nella Tabella 2 si riportano le prime 10 righe del risultato prodotto dal job. Poiché le tre implementazioni producono lo stesso schema e la stessa struttura logica dell'output, viene mostrato un unico esempio rappresentativo. Per motivi di leggibilità, alcuni valori della colonna *Top causes* sono stati omessi e sostituiti con il simbolo "...".

Origin	Month	Band	Flights	Avg Dep	Avg Arr	Top causes
ABE	1	low	45	6.53	-0.71	[{nas_delay, 7}, {carrier_delay, 4}, ...]
ABE	1	medium	30	34.9	32.5	[{late_aircraft_delay, 16}, {nas_delay, 14}, ...]
ABE	1	high	30	241.3	234.03	[{carrier_delay, 15}, ...]
ABE	1	cancelled	11	NULL	NULL	[{weather, 11}]
ABE	2	low	32	6.03	-6.09	[{nas_delay, 2}, {carrier_delay, 1}]
ABE	2	medium	19	28.53	10.11	[{carrier_delay, 6}, {late_aircraft_delay, 5}, ...]
ABE	2	high	14	227.93	217.93	[{carrier_delay, 8}, {late_aircraft_delay, 7}, ...]
ABE	2	cancelled	3	NULL	NULL	[{weather, 3}]
ABE	3	low	47	7.26	-6.57	[{late_aircraft_delay, 2}, {nas_delay, 2}, ...]
ABE	3	medium	28	30.68	19.89	[{late_aircraft_delay, 8}, {carrier_delay, 6}, ...]

Tabella 2: Report dei ritardi per aeroporto e mese (Task 3.2)

3.3 Ranking delle coppie compagnia-aeroporto con comportamento anomalo nei ritardi

La terza analisi ha l'obiettivo di valutare le performance operative delle compagnie aeree a livello di singolo aeroporto di partenza, confrontando il comportamento di ciascuna compagnia con quello medio dell'aeroporto stesso.

Per ogni coppia (aeroporto di partenza, compagnia aerea) vengono calcolate diverse metriche relative a ritardi e cancellazioni. Inoltre, per ogni aeroporto viene determinato il ritardo medio complessivo in partenza, che viene utilizzato come riferimento per confrontare le prestazioni delle singole compagnie e realizzare così un ranking.

3.3.1 Preparazione dei dati

Per l'esecuzione del job sono stati utilizzati i seguenti attributi del dataset preprocessato:

- aeroporto di origine (`origin`);
- compagnia aerea (`op_unique_carrier`);
- ritardo in partenza (`dep_delay`);
- ritardo in arrivo (`arr_delay`);
- flag di cancellazione del volo (`cancelled`).

3.3.2 Descrizione dell'analisi

Per ciascuna coppia aeroporto-compagnia vengono calcolate le seguenti metriche:

- numero totale di voli operati;
- ritardo medio in partenza;

- ritardo medio in arrivo;
- tasso di cancellazione;
- differenza tra il ritardo medio in partenza della compagnia e il ritardo medio complessivo dell'aeroporto;
- posizione della compagnia nel ranking dell'aeroporto basato sul ritardo medio in partenza.

Per ogni aeroporto viene quindi costruita una classifica delle compagnie presenti, ordinata in base al ritardo medio in partenza. La differenza rispetto alla media aeroportuale consente di evidenziare le compagnie che mostrano prestazioni migliori o peggiori rispetto al comportamento medio osservato nello stesso scalo.

Il risultato finale è organizzato per aeroporto e riporta, per ciascuna compagnia, le statistiche calcolate, la differenza rispetto alla media dell'aeroporto e la posizione occupata nel ranking.

Il risultato dell'analisi è rappresentato dalla seguente struttura dati:

```
root
|-- origin: string
|-- op_unique_carrier: string
|-- num_flights: long
|-- avg_dep_delay: double
|-- avg_arr_delay: double
|-- cancel_rate: double
|-- dep_delay_diff: double
|-- rank: integer
```

3.3.3 Implementazione con Spark Core

Nella prima fase, ogni record viene trasformato in una coppia chiave-valore, dove la chiave è definita dalla coppia (`origin`, `op_unique_carrier`), che identifica una combinazione aeroporto-compagnia. Il valore associato contiene le informazioni necessarie al calcolo delle metriche aggregate:

- numero totale di voli;
- somma dei ritardi in partenza;
- somma dei ritardi in arrivo;
- numero di valori validi per il calcolo dei ritardi medi in partenza e in arrivo;
- numero di voli cancellati.

La fase di aggregazione viene realizzata tramite l'operazione `reduceByKey()`, che combina iterativamente i valori associati alla stessa chiave. Durante questa operazione vengono aggiornate le somme dei ritardi, i conteggi necessari al calcolo delle medie, il numero totale di voli e il numero di cancellazioni.

Successivamente vengono calcolate le metriche finali per ciascuna coppia aeroporto-compagnia:

- numero totale di voli;
- ritardo medio in partenza;

- ritardo medio in arrivo;
- tasso di cancellazione.

Per il calcolo dei ritardi medi vengono considerati esclusivamente i valori non nulli disponibili nel dataset.

Successivamente viene determinato il ritardo medio complessivo in partenza per ciascun aeroporto, utilizzato come valore di riferimento per confrontare le compagnie che operano nello stesso scalo. Tale valore viene ottenuto tramite una seconda aggregazione `reduceByKey()`, che calcola la somma dei ritardi in partenza e il numero di osservazioni valide per ogni aeroporto.

Per ogni coppia aeroporto-compagnia viene quindi calcolata la differenza tra il ritardo medio della compagnia e il ritardo medio complessivo dell'aeroporto. Le compagnie vengono successivamente ordinate in base al ritardo medio in partenza, ottenendo così il ranking richiesto per ciascun aeroporto.

Infine, il risultato viene convertito nella struttura finale dell'analisi, ordinato per aeroporto e posizione nel ranking e salvato in formato Parquet.

3.3.4 Implementazione con Spark SQL

La pipeline è composta da cinque fasi principali: calcolo delle metriche per coppia aeroporto-compagnia, costruzione della media aeroportuale di riferimento, unione dei risultati, calcolo della differenza rispetto alla media e generazione del ranking.

Nella prima fase vengono calcolate le statistiche per ciascuna coppia (`origin`, `op_unique_carrier`) tramite un'operazione di aggregazione. Per ogni gruppo vengono determinati il numero totale di voli, il ritardo medio in partenza, il ritardo medio in arrivo e il tasso di cancellazione.

Successivamente viene calcolato, per ciascun aeroporto di origine, il ritardo medio complessivo in partenza considerando tutti i voli associati allo scalo. Questo valore rappresenta il riferimento rispetto al quale confrontare le prestazioni delle singole compagnie operanti nello stesso aeroporto.

Le due aggregazioni vengono quindi combinate tramite un'operazione di `JOIN` sulla colonna `origin`. Sul risultato ottenuto viene calcolata la differenza tra il ritardo medio in partenza della compagnia e il ritardo medio complessivo dell'aeroporto, permettendo di evidenziare quanto una compagnia si discosti dalla media dello scalo.

Infine viene costruito il ranking delle compagnie all'interno di ciascun aeroporto tramite una `Window function`, che consente di ordinare le compagnie sulla base del ritardo medio in partenza crescente. Utilizzando la funzione `row_number()`, viene assegnata una posizione progressiva all'interno di ogni aeroporto, identificando con rank 1 la compagnia con il minor ritardo medio.

Il risultato finale viene ordinato per aeroporto e posizione nel ranking e salvato in formato Parquet.

3.3.5 Implementazione con Hive

Nella prima fase vengono create due tabelle intermedie. La prima contiene le statistiche aggregate per ciascuna coppia aeroporto di partenza-compagnia, ottenute tramite un'operazione di `GROUP BY` su `origin` e `op_unique_carrier`. Per ogni gruppo vengono calcolati il numero totale

di voli, il ritardo medio in partenza, il ritardo medio in arrivo e il tasso di cancellazione.

La seconda tabella intermedia contiene invece il ritardo medio complessivo in partenza per ciascun aeroporto, calcolato tramite un'aggregazione su `origin`. Questo valore viene utilizzato come riferimento per confrontare le prestazioni delle singole compagnie rispetto al comportamento medio dello stesso aeroporto.

Successivamente le due tabelle vengono unite tramite un'operazione di `JOIN` sulla colonna `origin`. A partire dal risultato viene calcolata la differenza tra il ritardo medio in partenza della compagnia e il ritardo medio complessivo dell'aeroporto.

Nella fase finale viene applicata una funzione finestra tramite `ROW_NUMBER() OVER`, che assegna una posizione progressiva alle compagnie all'interno di ciascun aeroporto, ordinandole in base al ritardo medio in partenza crescente. In questo modo il rank 1 identifica la compagnia con il minor ritardo medio nello specifico aeroporto.

Il risultato viene infine salvato in formato Parquet e le tabelle intermedie utilizzate durante l'elaborazione vengono eliminate.

3.3.6 Risultato del job

Di seguito nella Tabella 3 si riporta un estratto del risultato finale prodotto dal job. Lo schema è identico per tutte le implementazioni.

Origin	Carrier	Flights	Avg Dep	Avg Arr	Cancel Rate	Dep Diff	Rank
ABE	G4	1839	8.43	0.64	0.0163	-3.73	1
ABE	9E	1017	9.42	0.67	0.0138	-2.74	2
ABE	OH	1166	15.55	4.45	0.0129	3.39	3
ABE	OO	315	30.63	22.31	0.0381	18.47	4
ABI	MQ	1757	7.26	4.41	0.0085	0.0	1
ABQ	DL	1547	1.75	-5.6	0.009	-7.69	1
ABQ	OO	3987	2.1	-2.05	0.0045	-7.34	2
ABQ	MQ	907	5.28	3.26	0.0066	-4.16	3
ABQ	UA	1822	5.48	-0.32	0.0104	-3.96	4
ABQ	AS	423	8.42	4.53	0.0095	-1.02	5

Tabella 3: Ranking compagnie per aeroporto basato sul ritardo medio in partenza (Task 3.3)

4 Valutazione delle prestazioni e risultati sperimentali

In questa sezione vengono riportati e discussi i risultati sperimentali ottenuti dalle implementazioni realizzate con Spark Core, Spark SQL e Hive. L'obiettivo è confrontare il comportamento delle diverse tecnologie al variare della dimensione del dataset e delle risorse computazionali disponibili.

4.1 Metriche di valutazione

Per il confronto delle prestazioni sono state considerate due metriche temporali:

- **Job duration:** tempo complessivo di esecuzione del job, comprensivo delle fasi di inizializzazione, scheduling, comunicazione tra componenti, operazioni di shuffle e overhead introdotti dal sistema;

- **Task time:** somma dei tempi di esecuzione delle singole task, utilizzata per valutare il tempo effettivamente impiegato nelle operazioni computazionali.

La *job duration* viene utilizzata come metrica principale per il confronto tra le diverse implementazioni, in quanto rappresenta il tempo necessario per completare l'elaborazione.

Il *task time* viene invece utilizzato come metrica complementare, utile per distinguere il costo computazionale dagli overhead introdotti dall'ambiente di esecuzione.

Per le implementazioni basate su Spark Core e Spark SQL, le metriche sono state estratte tramite Spark History Server, mentre per Hive sono state raccolte direttamente durante l'esecuzione dei job. I valori di *job duration* sono riportati in secondi interi, mentre i valori di *task time* sono riportati con una cifra decimale.

4.2 Task 3.1 - Tempi di esecuzione

4.2.1 Spark Core

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	12	36	33	35
50	17	42	38	36
75	23	52	42	43
100	28	54	49	45
200	48	90	78	57
400	90	145	103	80

Tabella 4: Task 3.1 - Spark Core - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	8.1	19.3	16.2	18.2
50	14.1	24.4	21.5	19.3
75	19.2	30.4	24.2	22.5
100	24.1	36.4	30.4	24.6
200	45.1	71.7	58.4	40.5
400	85.2	125.2	84.6	62

Tabella 5: Task 3.1 - Spark Core - Task time

Dai risultati ottenuti emerge che l'aumento della dimensione del dataset comporta un incremento del tempo di esecuzione in tutte le configurazioni considerate. In particolare, passando dal dataset al 25% a quello al 400%, la *job duration* in ambiente locale aumenta da 12 a 90 secondi, mentre sul cluster i migliori risultati si hanno con 6 nodi, in cui si passa da 35 a 80 secondi.

Per dataset di dimensioni ridotte l'esecuzione locale risulta nettamente più efficiente rispetto alle configurazioni distribuite. Ad esempio, sul dataset al 25% la configurazione locale richiede 12 secondi, mentre il cluster impiega tra i 33 ed i 36 secondi. Questo comportamento è dovuto principalmente all'overhead introdotto dall'esecuzione distribuita, legato all'avvio dei componenti Spark, alla gestione delle risorse, alla comunicazione tra nodi e alle operazioni di scheduling, che incide maggiormente quando il carico computazionale è limitato.

All'aumentare della dimensione dell'input, il vantaggio della distribuzione diventa progressivamente più evidente. Sul dataset al 400%, la configurazione con 6 nodi raggiunge una *job duration* di 80 secondi, risultando più veloce rispetto all'esecuzione locale (90 secondi) e alle configurazioni distribuite con un numero inferiore di nodi. Questo mostra come l'aumento delle risorse disponibili permetta di ridurre il tempo complessivo quando la quantità di dati è sufficiente a compensare l'overhead iniziale.

Analizzando il *task time* si osserva un comportamento simile, ma con differenze più evidenti legate al solo costo computazionale. Già sul dataset al 400%, la configurazione con 4 nodi (84.6 secondi) raggiunge prestazioni sostanzialmente comparabili a quelle dell'esecuzione locale (85.2 secondi), mentre quella con 6 nodi riduce ulteriormente il tempo fino a 62 secondi. Questo evidenzia che la suddivisione del lavoro tra più executor consente di ottenere benefici anche prima

rispetto a quanto osservato considerando la sola durata complessiva del job.

La differenza tra *job duration* e *task time* evidenzia il peso dell'overhead introdotto dall'ambiente distribuito: nelle configurazioni con dataset piccoli, il tempo aggiuntivo necessario alla gestione del cluster annulla i vantaggi della parallelizzazione, mentre su dataset più grandi la maggior parte del tempo è legata all'elaborazione effettiva dei dati.

Nel complesso, l'implementazione Spark Core mostra una buona scalabilità all'aumentare della dimensione dell'input. L'utilizzo di un numero maggiore di nodi non porta vantaggi significativi per dataset piccoli, ma permette una riduzione dei tempi per dataset più grandi, dove il costo delle operazioni distribuite viene compensato dal maggiore parallelismo.

Per visualizzare meglio l'andamento delle due metriche considerate, vengono riportati nelle Figure 1 e 2 anche i grafici relativi alla variazione della *job duration* e del *task time* al crescere della dimensione del dataset.

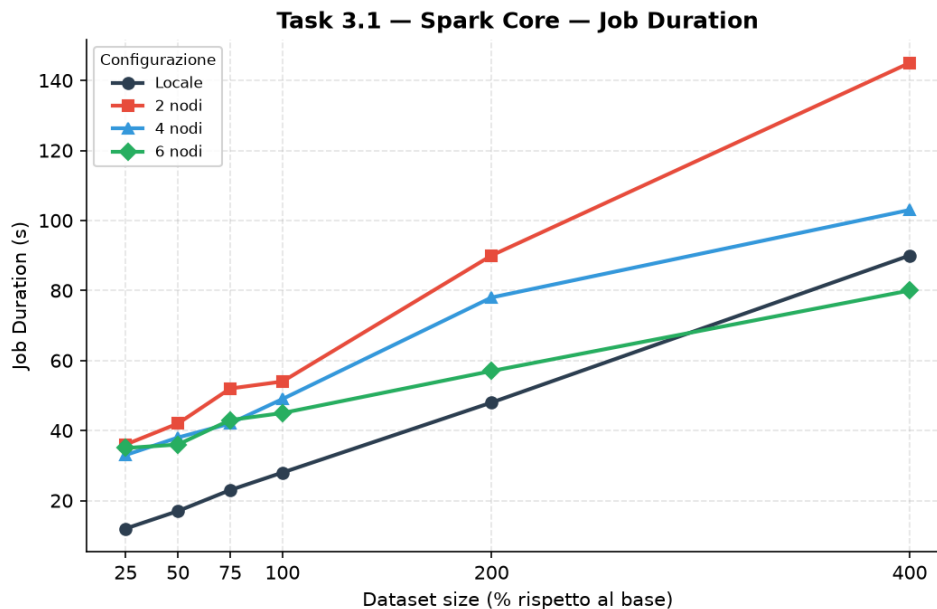


Figura 1: Task 3.1 - Spark Core - Andamento della job duration

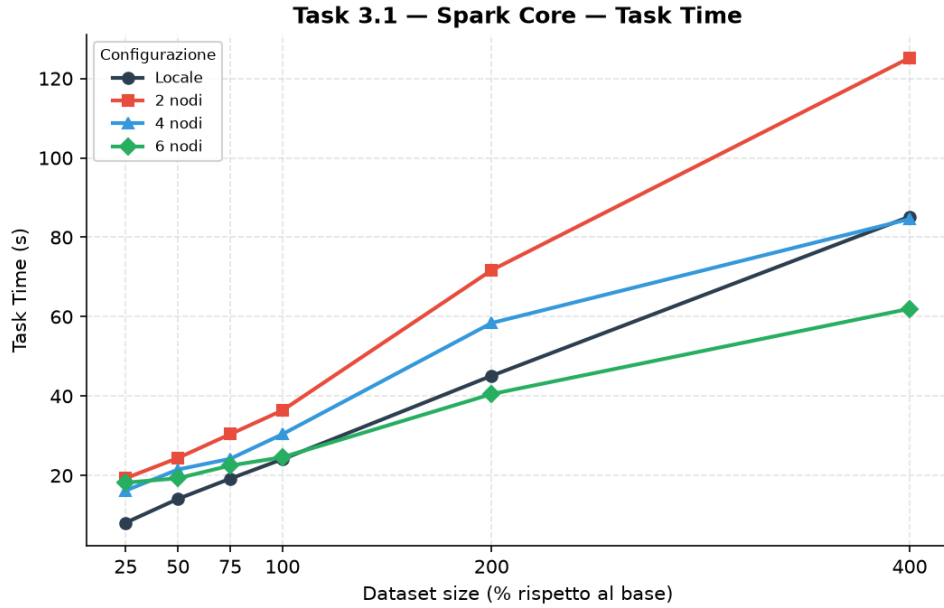


Figura 2: Task 3.1 - Spark Core - Andamento del task time

Dai grafici si può notare quindi come l'andamento della crescita sia della job duration che del task time risulti meno marcato nelle configurazioni distribuite rispetto all'esecuzione locale, in particolare nel caso di 4 nodi e 6 nodi. Nella configurazione con 2 nodi invece l'andamento segue una crescita molto simile a quella locale, evidenziando come in questo caso l'aumento delle risorse disponibili non sia sufficiente a compensare l'overhead introdotto dall'esecuzione distribuita.

4.2.2 Spark SQL

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	7	29	22	20
50	7	29	22	21
75	8	31	23	21
100	8	32	23	21
200	9	34	25	22
400	12	39	29	26

Tabella 6: Task 3.1 - Spark SQL - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	3.3	12.9	11.7	11.8
50	3.6	12.8	12.1	11.9
75	4.4	13.4	12.7	11.9
100	4.5	13.8	14.1	11.9
200	5.4	17	15	13
400	9.1	20.9	17.9	17.1

Tabella 7: Task 3.1 - Spark SQL - Task time

Dai risultati ottenuti emerge che anche in questo caso l'aumento della dimensione del dataset comporta un incremento dei tempi di esecuzione, sebbene in misura più contenuta rispetto a quanto osservato per l'implementazione con Spark Core descritta precedentemente. In particolare, la *job duration* in ambiente locale passa da 7 secondi sul dataset al 25% a 12 secondi sul dataset al 400%, mostrando una crescita limitata all'aumentare della quantità di dati elaborati.

A differenza di quanto osservato per Spark Core, l'esecuzione locale mantiene il vantaggio anche all'aumentare della dimensione del dataset, almeno per le dimensioni considerate nell'esperimento. Sul dataset al 400%, la *job duration* locale è pari a 12 secondi, mentre la migliore configurazione distribuita, ottenuta con 6 nodi, richiede 26 secondi. Questo evidenzia come, per questa specifica analisi, il beneficio ottenuto dal parallelismo non risulti sufficiente a compensare completamente l'overhead introdotto dall'esecuzione distribuita.

Analizzando il *task time*, il tempo effettivamente impiegato nelle operazioni computazionali cresce da 3.3 a 9.1 secondi in ambiente locale e da 11.8 a 17.1 secondi nella configurazione con 6 nodi. Anche in questo caso la configurazione locale mostra risultati migliori, mentre tra le configurazioni distribuite si osserva generalmente una riduzione dei tempi all'aumentare del numero di nodi disponibili.

Nel complesso, Spark SQL mostra prestazioni migliori rispetto a Spark Core per questa analisi. I tempi di esecuzione crescono in maniera più contenuta all'aumentare della dimensione del dataset e l'impatto della variazione dell'input risulta meno marcato, evidenziando un comportamento più stabile per questa tipologia di elaborazione.

Per una rappresentazione visiva dei risultati ottenuti, nelle Figure 3 e 4 sono riportati i grafici relativi alla *job duration* e al *task time*.

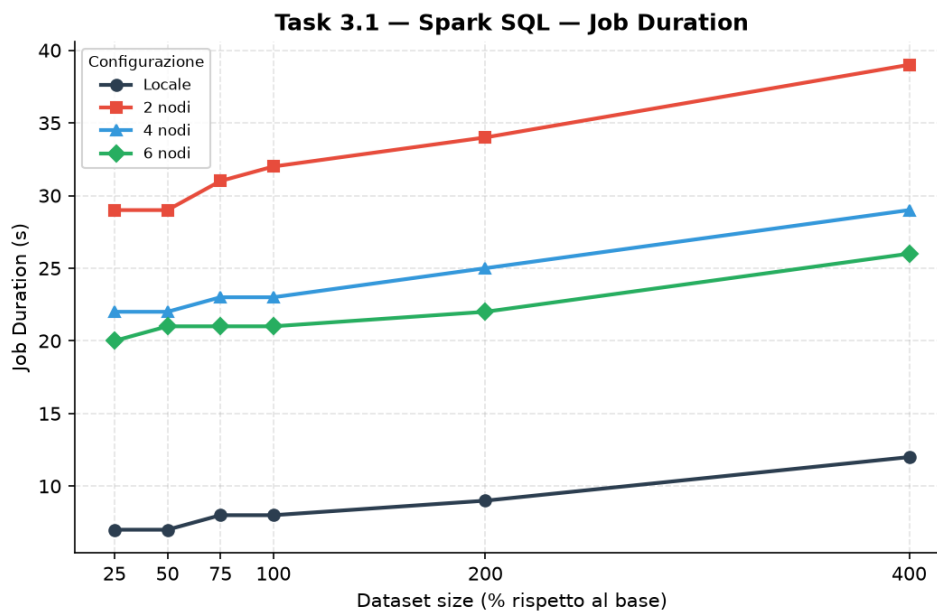


Figura 3: Task 3.1 - Spark SQL - Andamento della job duration

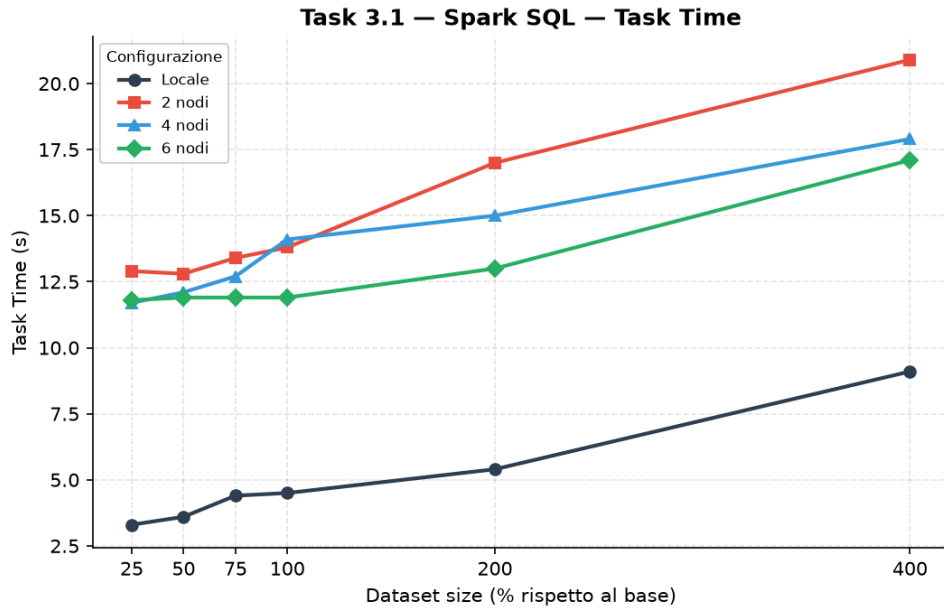


Figura 4: Task 3.1 - Spark SQL - Andamento del task time

4.2.3 Hive

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	12	40	32	43
50	13	44	43	37
75	17	46	37	38
100	16	45	37	38
200	22	49	44	46
400	34	53	51	50

Tabella 8: Task 3.1 - Hive - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	9.3	27.7	21.4	25.8
50	10.2	32.3	31.1	23.7
75	14.1	32.3	26.4	25.9
100	13.3	32.8	24.9	25.9
200	19.2	37.3	32.5	34.6
400	31.4	41.1	39.8	37.7

Tabella 9: Task 3.1 - Hive - Task time

Dai risultati ottenuti emerge che anche per Hive l'aumento della dimensione del dataset comporta un incremento dei tempi di esecuzione in tutte le configurazioni considerate. In particolare, la *job duration* in ambiente locale passa da 12 secondi sul dataset al 25% a 34 secondi sul dataset al 400%, mentre nella configurazione con 6 nodi varia da 43 a 50 secondi.

Per dataset di dimensioni ridotte, l'esecuzione locale risulta nuovamente più efficiente rispetto alle configurazioni distribuite. All'aumentare della dimensione dell'input, il divario tra ambiente locale e cluster tende a ridursi, ma per le dimensioni considerate nell'esperimento la distribuzione non porta ancora un vantaggio in termini di *job duration*. Sul dataset al 400%, infatti, la configurazione locale raggiunge 34 secondi, mentre la configurazione migliore tra quelle distribuite, ottenuta con 6 nodi, richiede 50 secondi.

Analizzando il *task time*, si osservano valori che vanno da 9.3 a 31.4 secondi in locale e da 25.8 a 37.7 secondi nella configurazione con 6 nodi. Anche considerando il solo tempo computazionale, l'esecuzione locale mantiene i risultati migliori; tuttavia, all'aumentare della dimensione del dataset, la differenza rispetto alla configurazione distribuita tende a ridursi sensibilmente.

Nel complesso, Hive mostra prestazioni intermedie rispetto alle altre implementazioni considerate. Rispetto a Spark Core presenta tempi più contenuti, ma non raggiunge le prestazioni

ottenute da Spark SQL. Per questa analisi l'esecuzione distribuita introduce un overhead che non viene completamente compensato dall'aumento del parallelismo. Tuttavia, la progressiva riduzione del divario tra configurazione locale e distribuita suggerisce che, su volumi di dati maggiori, il parallelismo potrebbe fornire benefici più evidenti.

Quanto descritto è riportato anche nelle Figure 5 e 6, dove vengono mostrati gli andamenti della *job duration* e del *task time* al variare della dimensione del dataset.

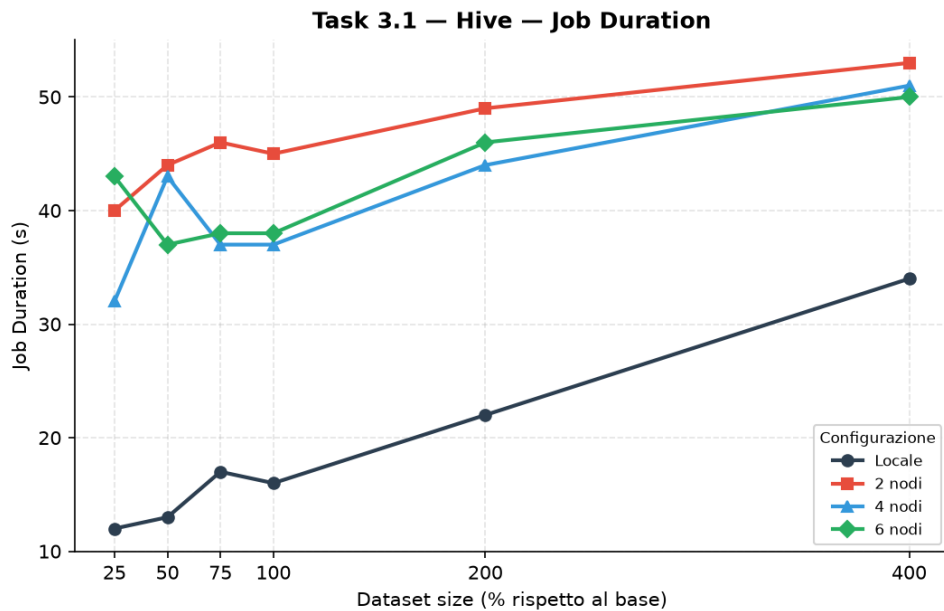


Figura 5: Task 3.1 - Hive - Andamento della job duration

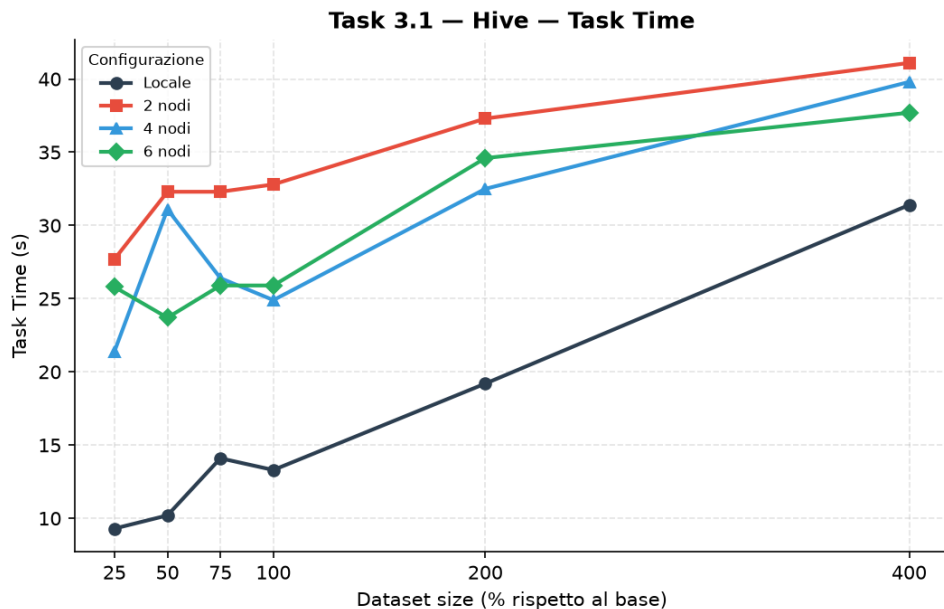


Figura 6: Task 3.1 - Hive - Andamento del task time

4.3 Task 3.2 - Tempi di esecuzione

4.3.1 Spark Core

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	11	34	32	31
50	16	41	41	34
75	21	46	43	38
100	26	53	45	42
200	44	82	62	54
400	84	132	95	77

Tabella 10: Task 3.2 - Spark Core - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	8	17.7	14.6	12.5
50	13	22.6	21.9	16.6
75	18.1	28.7	24.8	20.8
100	23	33.6	27.7	24.6
200	41	64.8	44.8	35.7
400	79.1	119.8	76.8	58

Tabella 11: Task 3.2 - Spark Core - Task time

Dai risultati ottenuti emerge che, rispetto alla configurazione locale, l'utilizzo del cluster consente di ottenere benefici più evidenti all'aumentare del carico elaborativo. Considerando la *job duration*, in ambiente locale il tempo passa da 11 secondi sul dataset al 25% a 84 secondi sul dataset al 400%, mentre tra le configurazioni distribuite il miglior comportamento si osserva con 6 nodi, dove il tempo varia da 31 a 77 secondi. In particolare, sul dataset al 400% la configurazione con 6 nodi risulta più efficiente rispetto all'esecuzione locale, in linea con quanto osservato nell'implementazione Spark Core della Task 3.1.

Analizzando il *task time*, il beneficio della distribuzione risulta ancora più evidente. Sul dataset al 400%, la configurazione con 4 nodi (76.8 secondi) risulta leggermente migliore rispetto all'esecuzione locale (79.1 secondi), mentre quella con 6 nodi riduce ulteriormente il tempo fino a 58 secondi.

Nel complesso, Spark Core mostra un comportamento coerente con quanto osservato nel Task 3.1: l'aumento delle risorse disponibili risulta efficace principalmente per dataset di dimensioni elevate, mentre per input ridotti il costo della distribuzione limita i benefici ottenibili.

Le considerazioni precedenti sono evidenziate anche nelle Figure 7 e 8, che mostrano l'andamento della *job duration* e del *task time* al variare della dimensione del dataset.

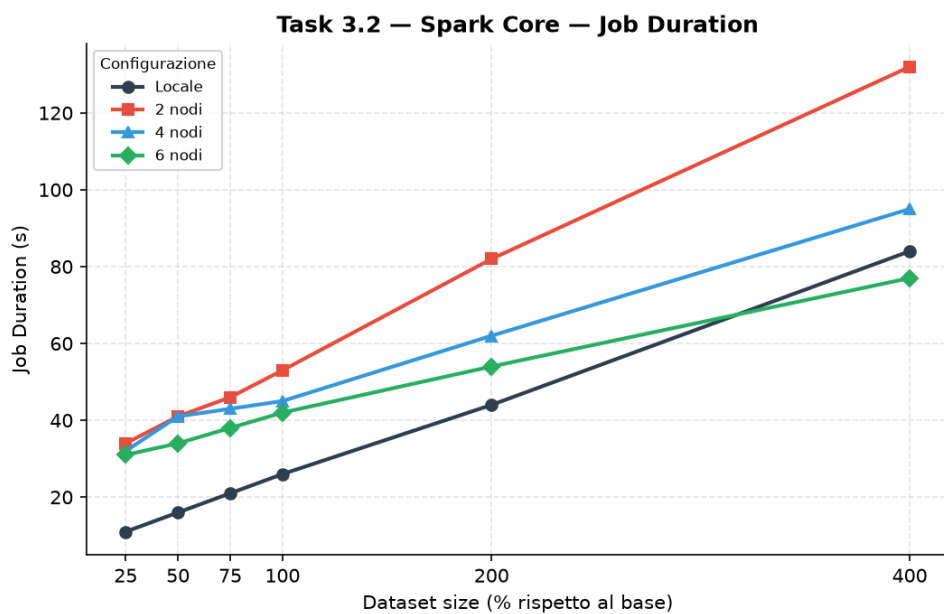


Figura 7: Task 3.2 - Spark Core - Andamento della job duration

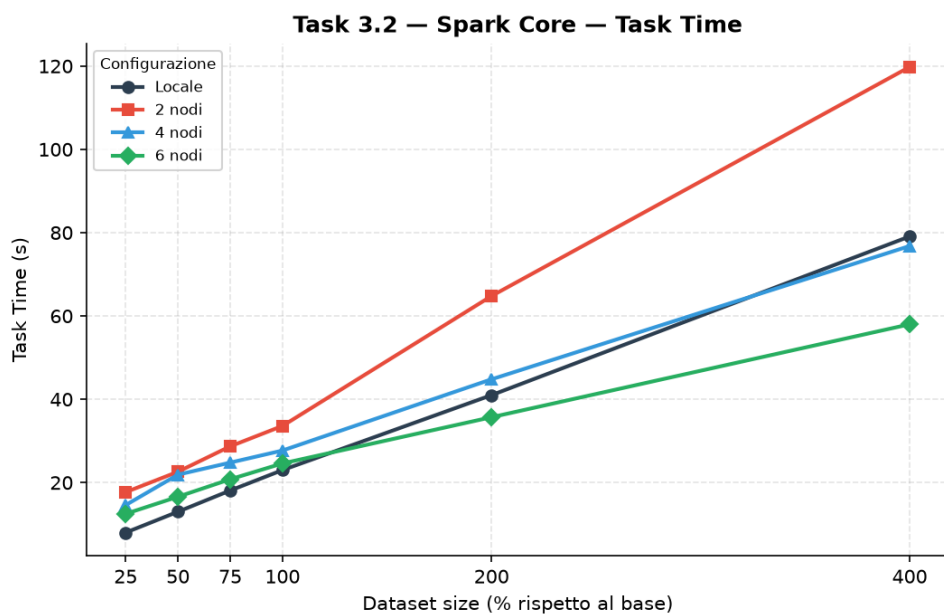


Figura 8: Task 3.2 - Spark Core - Andamento del task time

4.3.2 Spark SQL

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	8	28	22	20
50	8	30	23	21
75	8	31	22	21
100	9	31	23	21
200	9	30	24	22
400	11	32	25	26

Tabella 12: Task 3.2 - Spark SQL - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	6.9	17.9	13	11.7
50	7	18.9	14.4	12.8
75	7.1	18.9	14.5	12.6
100	8.4	19.2	16.1	12.7
200	8.5	21.8	16.3	14.6
400	10.2	25.9	17.6	17.8

Tabella 13: Task 3.2 - Spark SQL - Task time

Così come osservato nella Task 3.1, Spark SQL presenta un comportamento più stabile rispetto a Spark Core, con variazioni dei tempi di esecuzione molto contenute per le dimensioni di dataset considerate. Considerando la *job duration*, in ambiente locale si passa da 8 a 11 secondi dal dataset al 25% a quello al 400%. In particolare, per i dataset al 25%, 50% e 75% il tempo rimane pressoché invariato, indicando che per queste dimensioni l'esecuzione risulta poco influenzata dall'aumento della quantità di dati analizzata. Nelle configurazioni distribuite si osservano variazioni altrettanto ridotte, con i migliori risultati ottenuti con 6 nodi e con 4 nodi nel caso di dataset al 400%.

Per le dimensioni considerate, l'esecuzione locale risulta quella più efficiente. Ad esempio, sul dataset al 400% la configurazione locale richiede 11 secondi, mentre la migliore configurazione distribuita raggiunge 25 secondi con 4 nodi.

Analizzando il comportamento delle configurazioni distribuite si osserva comunque un miglioramento all'aumentare delle risorse disponibili. Passando da 2 a 6 nodi, infatti, la *job duration* si riduce da 28 a 20 secondi sul dataset al 25% e da 32 a 26 secondi sul dataset al 400%. Tuttavia, tale riduzione non è sufficiente a superare le prestazioni ottenute in ambiente locale.

Il *task time* mostra un andamento coerente con la *job duration*: il tempo computazionale cresce solo leggermente all'aumentare della dimensione del dataset, passando da 6.9 a 10.2 secondi in locale. Anche nelle configurazioni distribuite si osserva una riduzione dei tempi all'aumentare del numero di nodi, con il miglior risultato generalmente ottenuto dalla configurazione con 6 nodi.

Nel complesso, quindi, Spark SQL mantiene anche per questa task prestazioni migliori rispetto a Spark Core, mostrando una minore sensibilità alla variazione della dimensione dell'input e tempi di esecuzione inferiori.

Quanto appena detto viene mostrato anche nelle Figure 9 e 10, dove vengono riportati gli andamenti della *job duration* e del *task time* al variare della dimensione del dataset.

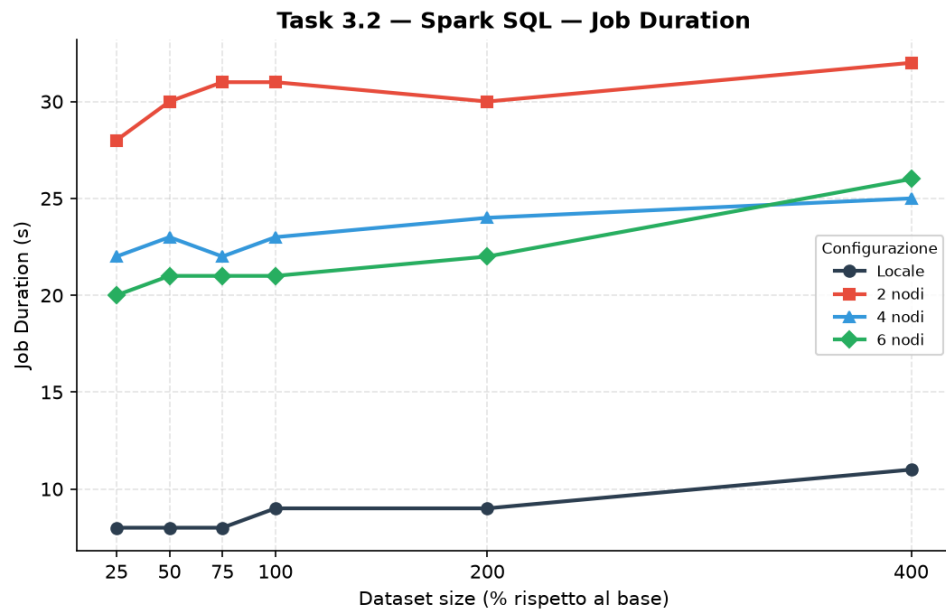


Figura 9: Task 3.2 - Spark SQL - Andamento della job duration

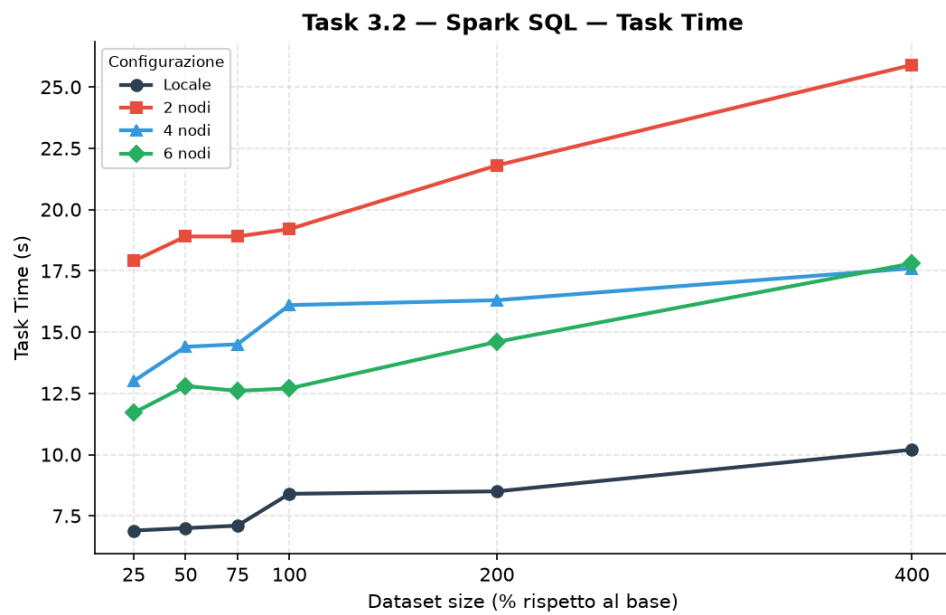


Figura 10: Task 3.2 - Spark SQL - Andamento del task time

4.3.3 Hive

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	24	59	55	54
50	27	55	52	56
75	30	55	57	58
100	35	61	63	57
200	49	62	57	65
400	76	69	63	62

Tabella 14: Task 3.2 - Hive - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	21.1	46.9	42.6	42.0
50	23.9	42.3	41.2	43.2
75	26.9	43.1	45.9	46.9
100	31.9	48.2	50.5	44.7
200	46.1	48.7	45.1	53.1
400	73.7	56.3	51.9	50.9

Tabella 15: Task 3.2 - Hive - Task time

Dai risultati ottenuti emerge un comportamento analogo a quello osservato per Spark Core nella Task 3.2, in cui le configurazioni distribuite tendono progressivamente a diventare più vantaggiose all'aumentare della quantità di dati elaborati. Rispetto alla Task 3.1 con Hive, in questa seconda analisi il beneficio dell'esecuzione distribuita risulta più evidente per dataset di dimensioni elevate, dove il parallelismo consente di ottenere tempi inferiori rispetto all'esecuzione locale.

Considerando la *job duration*, in ambiente locale il tempo passa da 24 secondi sul dataset al 25% a 76 secondi sul dataset al 400%, mostrando una crescita più marcata rispetto alla configurazione con 6 nodi, dove l'aumento del tempo risulta più contenuto, passando da 54 a 62 secondi.

Per dataset di dimensioni ridotte l'esecuzione locale risulta ancora la configurazione più efficiente, mentre all'aumentare della dimensione dell'input il comportamento cambia progressivamente. Sul dataset al 400%, infatti, la configurazione locale raggiunge 76 secondi, mentre la configurazione migliore su cluster, ottenuta con 6 nodi, richiede 62 secondi.

Analizzando il *task time*, questo comportamento risulta ancora più evidente: in ambiente locale il tempo computazionale cresce in modo simile alla *job duration*, mentre nelle configurazioni distribuite i valori risultano inferiori per dataset di grandi dimensioni, evidenziando il beneficio dell'elaborazione parallela.

Le differenze tra le configurazioni considerate sono evidenziate anche nelle Figure 11 e 12, dove vengono riportati gli andamenti della *job duration* e del *task time* al variare della dimensione del dataset.

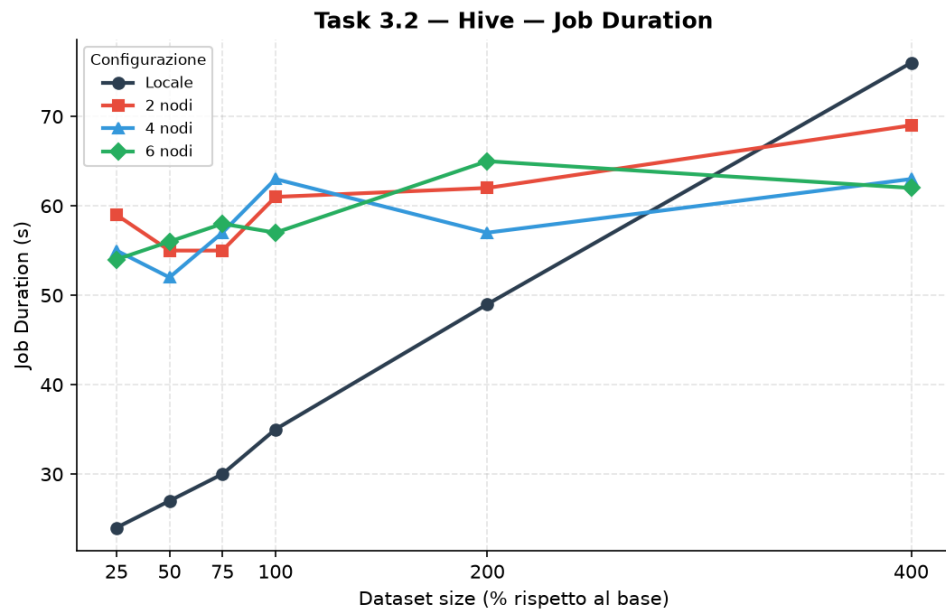


Figura 11: Task 3.2 - Hive - Andamento della job duration

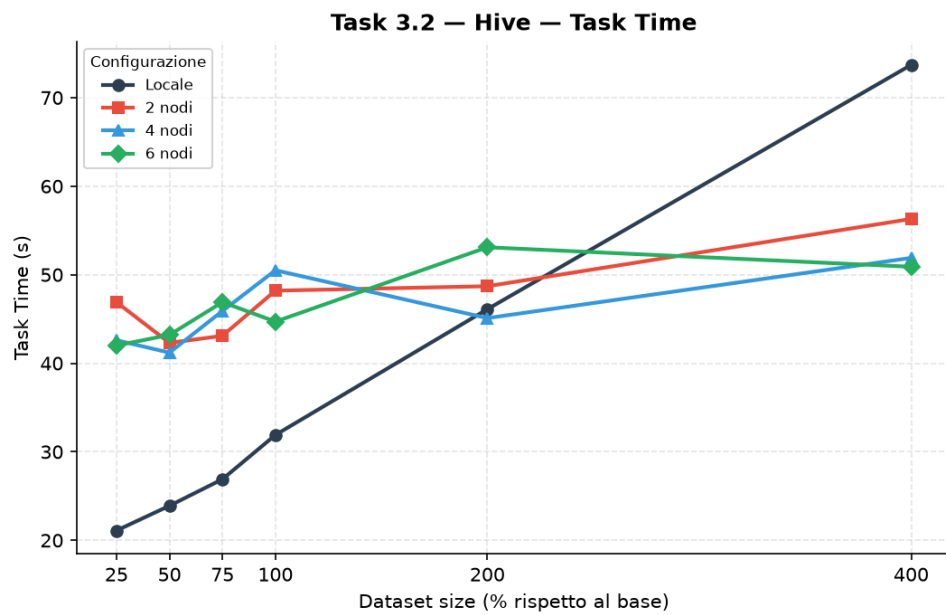


Figura 12: Task 3.2 - Hive - Andamento del task time

4.4 Task 3.3 - Tempi di esecuzione

4.4.1 Spark Core

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	16	40	40	35
50	25	51	45	39
75	35	60	53	47
100	45	74	62	53
200	84	124	108	75
400	162	214	147	111

Tabella 16: Task 3.3 - Spark Core - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	12.1	23.1	20.9	15.9
50	22	32	26.7	22.8
75	32	44.1	35.9	27.9
100	42.1	54.7	42	35.2
200	81.1	105.7	90.8	55.9
400	157.1	196.9	140.9	94.1

Tabella 17: Task 3.3 - Spark Core - Task time

Dai risultati ottenuti emerge come, tra le implementazioni analizzate, Spark Core nella Task 3.3 sia quella in cui il beneficio dell'esecuzione distribuita risulta più evidente all'aumentare del carico elaborativo. Considerando la *job duration*, in ambiente locale il tempo cresce da 16 secondi sul dataset al 25% a 162 secondi sul dataset al 400%, mentre nella configurazione con 6 nodi passa da 35 a 111 secondi.

Sul dataset al 400% la configurazione con 6 nodi raggiunge quindi 111 secondi, risultando sensibilmente migliore rispetto all'esecuzione locale (162 secondi) e alle configurazioni con un numero inferiore di nodi. Per dataset di dimensioni più ridotte, invece, il vantaggio della distribuzione risulta meno evidente considerando la sola *job duration*, poiché il tempo complessivo rimane influenzato dalla gestione dell'esecuzione su cluster.

Analizzando il *task time*, il beneficio del parallelismo risulta evidente già per dataset di dimensioni intermedie. A partire dal dataset al 50%, infatti, la configurazione con 6 nodi presenta un tempo computazionale molto vicino a quello dell'esecuzione locale (22.8 secondi contro 22 secondi), mostrando come la suddivisione del carico tra più nodi permetta di ridurre il tempo effettivamente dedicato all'elaborazione anche prima che il vantaggio sia visibile sulla durata complessiva del job. Sul dataset al 400%, la configurazione con 6 nodi riduce il tempo computazionale fino a 94.1 secondi, rispetto ai 157.1 secondi dell'esecuzione locale e ai 140.9 secondi della configurazione con 4 nodi.

Nel complesso, Spark Core nella Task 3.3 evidenzia come l'aumento delle risorse disponibili diventi particolarmente efficace all'aumentare della dimensione del dataset. L'utilizzo di un maggior numero di nodi consente infatti di sfruttare il parallelismo in modo più efficace, ottenendo tempi inferiori rispetto all'esecuzione su singolo nodo.

Le differenze tra le configurazioni considerate sono evidenziate anche nelle Figure 13 e 14, dove vengono riportati gli andamenti della *job duration* e del *task time*.

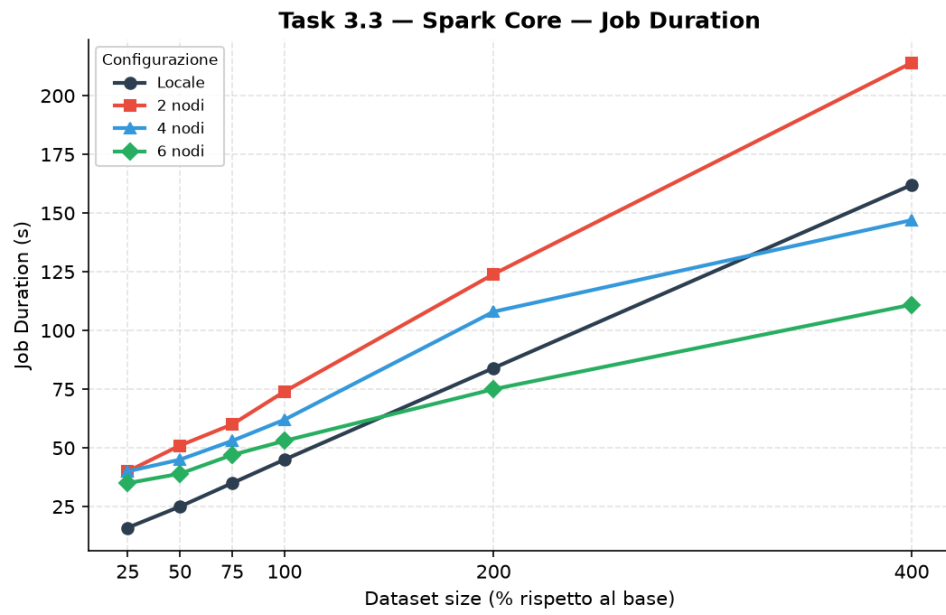


Figura 13: Task 3.3 - Spark Core - Andamento della job duration

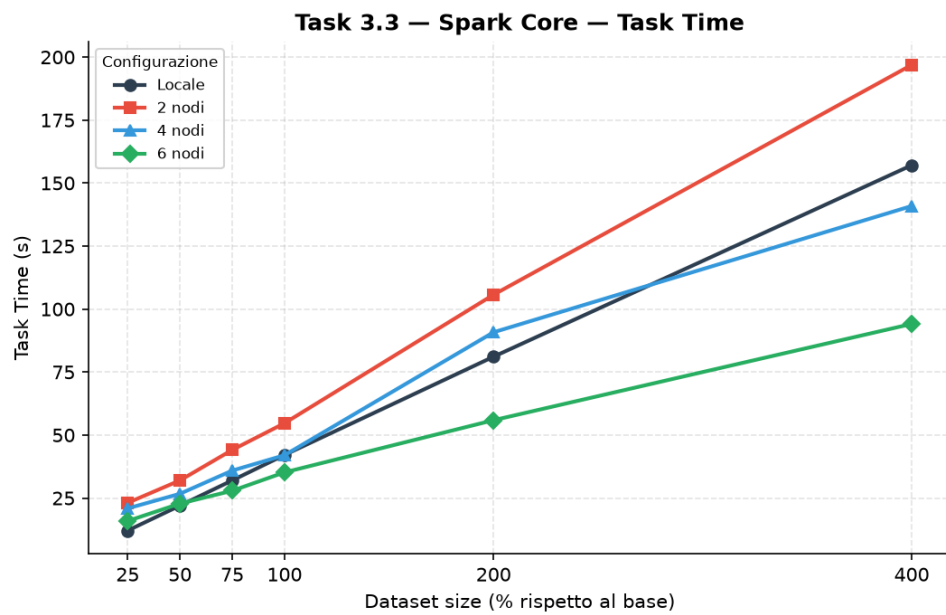


Figura 14: Task 3.3 - Spark Core - Andamento del task time

4.4.2 Spark SQL

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	6	27	19	18
50	6	27	19	18
75	6	28	19	18
100	6	27	20	18
200	6	27	20	19
400	7	29	21	21

Tabella 18: Task 3.3 - Spark SQL - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	4.1	12.6	13	13.6
50	3.2	12.3	14.4	12.9
75	4.1	12.5	14.5	13.3
100	4.1	13.6	16.1	13.7
200	4.2	14.6	16.3	14
400	5.2	17.4	17.6	15.8

Tabella 19: Task 3.3 - Spark SQL - Task time

Così come osservato nelle configurazioni precedenti, Spark SQL mantiene un comportamento più stabile rispetto a Spark Core, con tempi di esecuzione poco sensibili all'aumento della dimensione del dataset. Considerando la *job duration*, in ambiente locale il tempo rimane praticamente costante, passando da 6 secondi sul dataset al 25% a 7 secondi su quello al 400%. Anche nelle configurazioni distribuite le variazioni risultano contenute, con i migliori risultati ottenuti generalmente dalla configurazione con 6 nodi, che mantiene tempi compresi tra 18 e 21 secondi. Per le dimensioni considerate, l'esecuzione locale risulta quindi ancora la configurazione più efficiente.

Analizzando il *task time*, il comportamento risulta coerente con quello osservato per la *job duration*: il tempo computazionale in ambiente locale cresce solo leggermente, passando da 4.1 a 5.2 secondi. Anche nelle configurazioni distribuite la crescita dei valori rimane contenuta, indipendentemente dalla configurazione considerata.

Nel complesso, Spark SQL conferma anche nella Task 3.3 il vantaggio rispetto a Spark Core, mostrando tempi di esecuzione significativamente inferiori e una maggiore stabilità all'aumentare della dimensione dell'input.

Quanto appena detto viene mostrato anche nelle Figure 15 e 16, dove vengono riportati gli andamenti della *job duration* e del *task time* al variare della dimensione del dataset.

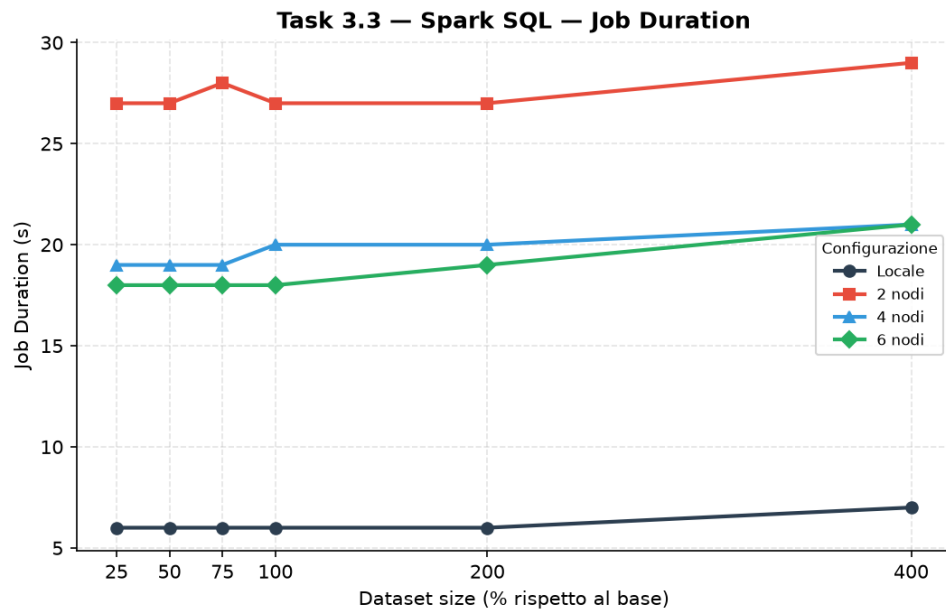


Figura 15: Task 3.3 - Spark SQL - Andamento della job duration

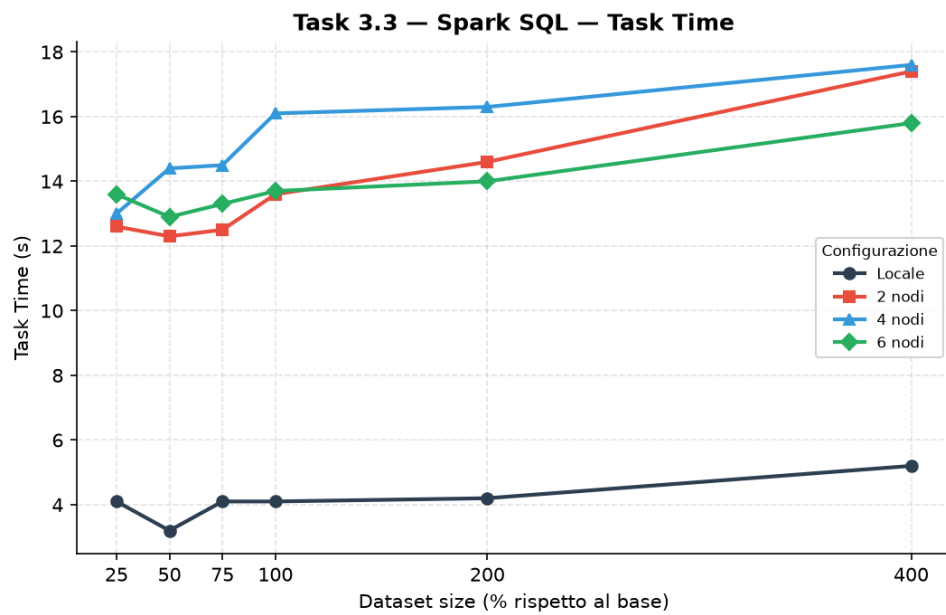


Figura 16: Task 3.3 - Spark SQL - Andamento del task time

4.4.3 Hive

Job duration (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	20	39	31	32
50	20	44	35	36
75	22	44	40	35
100	23	44	41	35
200	29	47	43	45
400	40	50	45	49

Tabella 20: Task 3.3 - Hive - Job duration

Task time (s)				
Dataset	Locale	2 nodi	4 nodi	6 nodi
25	17.7	25.9	19.8	19.2
50	17.6	31.8	24.4	23.1
75	19.7	31.7	28.2	22.8
100	20.7	31.8	29.8	23.4
200	26.6	33.9	32.2	33.9
400	37.7	38.2	34.2	37.2

Tabella 21: Task 3.3 - Hive - Task time

Dai risultati ottenuti emerge che Hive nella Task 3.3 presenta un comportamento più stabile rispetto a Spark Core, ma mantiene una maggiore sensibilità all'aumento della dimensione del dataset rispetto a Spark SQL. Considerando la *job duration*, in ambiente locale il tempo passa da 20 secondi sul dataset al 25% a 40 secondi sul dataset al 400%, mentre nelle configurazioni distribuite i valori rimangono compresi tra 31 e 50 secondi. La configurazione con 4 nodi presenta il comportamento più equilibrato, con tempi compresi tra 31 e 45 secondi, mentre quella con 6 nodi ottiene i risultati migliori sui dataset intermedi, senza però mantenere un vantaggio sul dataset più grande.

Rispetto alla Task 3.2 con Hive, in questa configurazione il vantaggio dell'esecuzione distribuita risulta meno evidente: mentre nella Task 3.2 il cluster riusciva a superare l'esecuzione locale sui dataset di grandi dimensioni, nella Task 3.3 le configurazioni distribuite raggiungono prestazioni molto simili senza però ottenere un miglioramento significativo. Sul dataset al 400%, infatti, l'esecuzione locale richiede 40 secondi, mentre la migliore configurazione distribuita raggiunge 45 secondi con 4 nodi. Questo mostra come l'aumento delle risorse permetta di ridurre il divario rispetto all'esecuzione locale, ma non sia sufficiente a garantire un vantaggio in termini di *job duration*.

Analizzando il *task time*, il comportamento risulta coerente con quello osservato per la *job duration*. In ambiente locale il tempo cresce da 17.7 a 37.7 secondi, mentre nelle configurazioni distribuite si hanno valori tra i 19.8 e i 38.2 secondi. Sul dataset al 400%, la configurazione con 4 nodi raggiunge il miglior risultato con 34.2 secondi, risultando leggermente migliore rispetto all'esecuzione locale (37.7 secondi). Il beneficio ottenuto non è però sufficiente a determinare un miglioramento evidente sul tempo complessivo del job, come invece osservato nella Task 3.2 di Hive, almeno per dataset di queste dimensioni.

Nel complesso, Hive nella Task 3.3 mostra un comportamento intermedio tra Spark Core e Spark SQL: rispetto a Spark Core presenta una crescita dei tempi più contenuta, mentre rispetto a Spark SQL risulta maggiormente influenzato dall'aumento della dimensione dell'input. L'esecuzione distribuita permette di ottenere benefici sul tempo computazionale, ma nelle dimensioni analizzate non è sufficiente a superare in modo evidente l'esecuzione locale.

Quanto appena detto viene mostrato anche nelle Figure 17 e 18, dove vengono riportati gli andamenti della *job duration* e del *task time* al variare della dimensione del dataset.

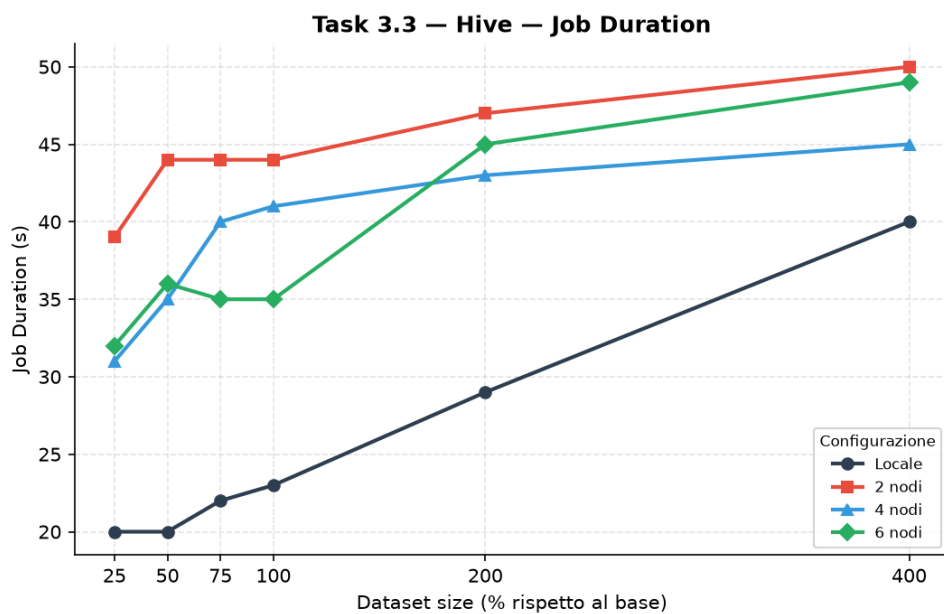


Figura 17: Task 3.3 - Hive - Andamento della job duration

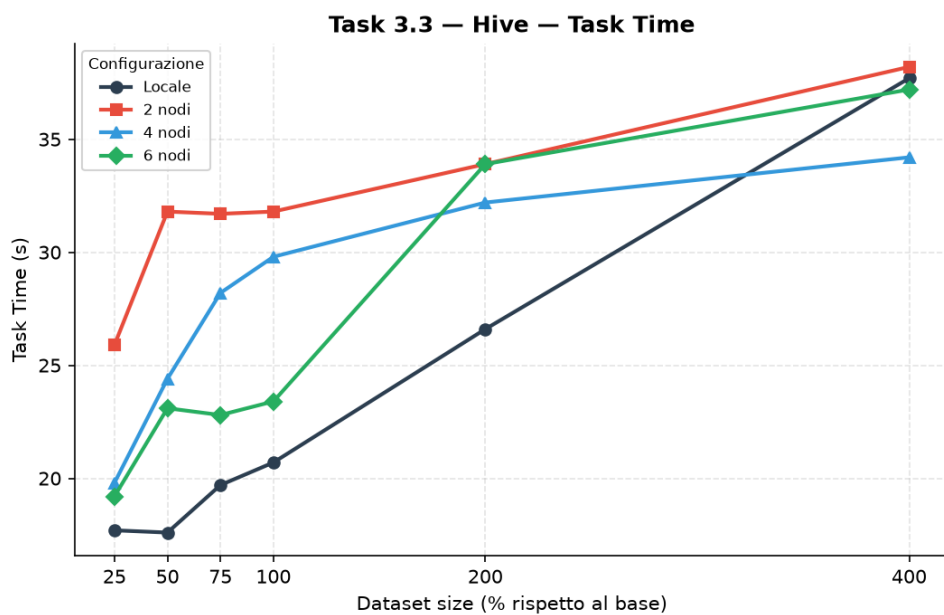


Figura 18: Task 3.3 - Hive - Andamento del task time

5 Discussione critica dei risultati

L'analisi sperimentale svolta nelle sezioni precedenti ha evidenziato differenze significative tra le tecnologie considerate e tra le diverse modalità di esecuzione.

In questa sezione vengono sintetizzate le principali osservazioni emerse, discutendo gli aspetti legati all'espressività, alla semplicità implementativa, all'efficienza e alla scalabilità delle soluzioni adottate.

5.1 Espressività e semplicità implementativa

Dal punto di vista dell'espressività, Spark SQL e Hive permettono di descrivere le analisi considerate attraverso un approccio dichiarativo basato su query. Le operazioni richieste dai job, come filtraggio, aggregazione e raggruppamento, possono essere espresse in modo compatto, riducendo la quantità di logica implementativa necessaria.

Spark Core richiede invece una definizione più esplicita del flusso di elaborazione, poiché le operazioni devono essere costruite manualmente attraverso trasformazioni sugli RDD. Questo comporta una maggiore complessità nello sviluppo, ma permette un controllo più diretto sulla pipeline di elaborazione e sulle operazioni applicate ai dati.

5.2 Efficienza e scalabilità

Come osservato anche nelle analisi precedenti, le tecnologie considerate presentano comportamenti differenti in termini di prestazioni e capacità di scalare su cluster. In particolare, dai risultati ottenuti emerge come Spark SQL sia generalmente la soluzione più efficiente per le analisi considerate, mostrando tempi di esecuzione inferiori rispetto alle altre tecnologie nelle configurazioni sperimentate.

Spark Core mostra invece una maggiore capacità di sfruttare l'aumento delle risorse disponibili. In particolare, nella Task 3.3 il beneficio dell'esecuzione distribuita risulta più evidente: all'aumentare della dimensione del dataset, la configurazione con più nodi permette di ridurre progressivamente il divario rispetto all'esecuzione locale, fino a ottenere prestazioni migliori sui dataset di dimensioni maggiori.

Hive presenta un comportamento intermedio tra Spark Core e Spark SQL. Rispetto a Spark SQL risulta maggiormente influenzato dalla dimensione dell'input, mentre rispetto a Spark Core mostra generalmente una crescita dei tempi più contenuta.

5.3 Impatto delle operazioni distribuite

Le prestazioni osservate dipendono non solo dalla tecnologia utilizzata, ma anche dal tipo di operazioni richieste dall'elaborazione. In particolare, operazioni di aggregazione e raggruppamento, ampiamente utilizzate nelle analisi considerate, possono richiedere fasi di shuffle dei dati tra i nodi del cluster, introducendo costi aggiuntivi di comunicazione e sincronizzazione.

L'analisi dei risultati mostra come l'utilizzo di un ambiente distribuito non comporti automaticamente un miglioramento delle prestazioni. Il vantaggio dell'esecuzione su cluster dipende infatti dalla capacità del carico di lavoro di sfruttare il parallelismo disponibile e di compensare i costi introdotti dalla distribuzione.

Di conseguenza, la scelta tra esecuzione locale e distribuita deve essere valutata in funzione

delle caratteristiche dell'elaborazione considerata: per analisi leggere o su dataset contenuti, una configurazione locale può risultare più efficiente grazie al minor overhead; al contrario, per elaborazioni più complesse o su quantità elevate di dati, l'approccio distribuito permette di ottenere benefici maggiori grazie alla suddivisione del carico computazionale.

6 Repository del progetto

Il codice sorgente utilizzato per la realizzazione delle analisi e degli esperimenti descritti in questo report è disponibile nella repository GitHub del progetto:

`https://github.com/francesco-pittacolo/big-data-flight-analysis`